Introduction to C: Module 1

Weekly Reading:

- Beej Guide, Chapter 2,
- "Should You Learn C To Learn How the Computer Works?" by Steve Klabnik.

Why Learn C?

You will almost certainly encounter C if you:

- do any systems programming.
- need to write high-performance software.
- want to know why C++, Java, and even Python are the way that they are.
- have to debug a low-level issue.
- study operating systems or computer architecture.

C is far from perfect, and we will explore many of its weaknesses here, but it's been the *lingua franca* for systems programming for four decades, and that's unlikely to change any time soon. There are wide swaths of computer science that require a working knowledge of it.

You may have heard that Python “is slow” and that C “is fast.” Well... sort of. There are dozens of factors that influence software performance. It is more accurate to say that C gives you control. It allows (and forces) you to decide what data structures to use, which algorithms to select, and when to free memory. These details are often hidden from you by higher-level languages, but when you need the kind of control that high-performance code requires, it's hard to beat C.

Is C difficult? Yes. Is it warty? Also yes. The good news is that it isn’t a very big language—within the first five weeks of this course, you’ll have enough intuition to build, in C, most of the projects you’ve done in higher-level languages. After that, we’ll focus on refinements—techniques and patterns, so you recognize them and understand the praxis of this language.

What This Course Is Not

This is not a C++ course. C++ is a radically different language with so many features (some ferociously ill-considered) that it would take multiple semesters to explore it all.

This is not a systems programming course—if the field interests you, consider taking CS 6200 (“Graduate Introduction to Operating Systems”) and CS 6210 (“Advanced Operating Systems”).

This is not a software engineering course. In fact, we'll explore some features that should rarely, if ever, be used, but that merit note for their unusual history or connections to important concepts in programming.

This is not an introductory programming course—on the contrary, it assumes familiarity with a higher-level language such as Java or Python, and you will find the course project difficult if you have not built anything of moderate scale (1000+ LoC) before.

This course is designed for intermediate or advanced programmers. My goal is that you learn C well enough that, when you take courses in the future that require it, the language itself is not a stumbling block.

Danger! C Assumes You Know What You're Doing...

... and this should probably never be assumed about anyone, ever. The fact that you're voluntarily taking a course on a hard programming language means that you're probably in the top 10% of programmers. But you create bugs. I create bugs. We all do.

C is, by design, so close to the hardware that familiar protective abstractions are not in place. C lets you do things that are weird and unsafe that higher-level languages would never allow.

Let me be concrete. If you're working in Python and have a list—or, in Java, an array—called `b` of length 10, and you try to write to `b[137]`, what happens? You get an exception. Usually, this crashes the program. This is what you want, most of the time, because the generation of a bad index means something has gone seriously wrong. If your shopping list has 10 items, “the 137th item” on it is nonsense.

C, on the other hand, allows you to write to the memory address that `b[137]` *would have, if it existed*. This is almost never a good thing to do, so why does C allow it?

Java and Python have decided that you must check your bounds—knowing you may forget, those languages do it for you. C does not—C’s attitude is that, if you want the checks done, write them yourself. If you’re unsure whether your index is in bounds, you must check it. But consider the two programs below:

```
int LENGTH = 100000000;
int a[LENGTH];
for (int i = 0; i < LENGTH; i++) {
    a[i] = i;
}
```

versus...

```
int LENGTH = 100000000;
```

```
int a[LENGTH];
for (int i = 0; i < LENGTH; i++) {
    if (i >= 0 && i < LENGTH) {
        a[i] = i;
    }
}
```

The second version is inefficient. We know, in this particular example, that the check will always pass, and we can safely delete it.

On the other hand, if `i` were an index derived from user input, we'd want to keep the check in place. Users, as we'll discuss in Module 6, should never be trusted. Tradeoffs, such as that between performance and safety, exist in all languages; C allows you to decide where, on each continuum, you want to be.

Compile and Run Time

C is a **compiled language**. Interpreted languages like Python and Clojure use compilation to improve performance, but there is a fundamental difference in philosophy between these "interpreted languages that also compile" and a language like C. The C compiler is not obligated to implement your program as you have written it. Let's say you've written a squaring function:

```
int square(int x) {
    return x * x;
}
```

and somewhere later you invoke it:

```
    ...
    int b = square(17);
    ...
```

In other languages, you can be certain that your `square` function will be called. However, the C compiler, when it generates executable code, has the right to skip it—for example, it could *inline* it to avoid the cost of a function call:

```
    ...
    int b = 17 * 17;
    ...
```

and then it could replace the constant expression with its result:

```
    ...
    int b = 289;
```

```
...
```

And if `b` is never used by the program, at all, it could be deleted outright. In other words, the compiler is allowed to improve your code in ways you didn't ask for, as long as the semantics don't change. This is almost never a problem, though it can cause headaches with certain classes of bugs, as we'll discuss later on.

There is no interactive toploop (or REPL) for C. To achieve interactivity, or to probe language features—you'll be doing a lot of this—you will have to get comfortable with the compiler. Start with small programs.

Some Advice

- **Compile often.** Make a small change, and compile. Make another small change, and compile. Code that usefully does something is closer to your target than code that doesn't compile.
- **Start at the first compilation error.** The compiler may give you dozens of errors, scaring you about the state of your program. As a general rule, focus on the first one—the one that threw the compiler off. The rest of the code may be correct.
- **Treat warnings as errors.** If the compiler says, "You shouldn't do this," it is almost always correct. Warnings often mean that the compiler has recognized a runtime crash—or worse, undefined behavior. Always aim for your programs to compile without warnings.
- **`printf` is your friend.** We'll discuss how to use debuggers (such as `gdb` and `lldb`) in Module 8, but `printf` suffices for small projects, and you should get comfortable using it to debug.
- **Test early and often.** The compiler will check some of your errors, but not all of them.

AI Policy—DITFOW (Important!)

You will use artificial intelligence throughout your career. I use it every day. Course policies vary. For your coursework here, ***it is 2021.*** Don't use AI for it. You should be able to do everything by hand—the reading, the coding, the debugging, the writing. You might never need to use GDB to debug software again, but that skill is what this course intends to teach, and you are learning it for reasons beyond the skill itself. Do It The Familiar Old Way. DITFOW.

If you're not able to complete the assignments using the materials provided, I ought to know that, so I can improve them. Feel free to ask questions on Ed, or reach out to me personally.

Exception 1: Although you are never graded on grammar, spelling, or usage, the use of AI-powered spelling- and grammar-checkers (e.g., Grammarly) is permitted.

Exception 2: In office hours and on Ed, discussions *about* AI (large language models, agentic tools, vibe coding, etc.) are permitted.

The Project

This course is project-based—you will be building larger and larger C projects, up to an interpreter for a bare-bones Lisp. This project's purpose is to give you intuition for the sorts of work that, when you use a higher-level language like Python, the runtime does to support you. Why is dynamic typing slow? What is garbage collection and why is it important? You will learn all of this.

To get full marks on the course project may require as much effort as your 3-credit courses do. If you can't get all the features to work, let me know what you're having trouble with. Communicate frequently, because if you're struggling with something, a lot of people are, too.

Office Hours

We will meet on **Mondays at 8:00pm Eastern time** using Zoom—your Canvas calendar should include the link. Attendance is not mandatory, and office hours will be recorded. When Monday is a holiday, office hours will be moved to Wednesday. For example, the Spring 2026 first office hour will be on **May 27**; the one after that will be on **June 1**.

Textbook

We'll be referring to Beej's guide to C Programming, available for free here. Only free resources will be used for the course.

Getting Started

You can use any operating system for this course. However, we will be assuming a Linux or Linux-like command-line environment. C is a compiled language, and you will be compiling and running code at the command line, as well as writing (simple) Makefiles later on. You can use any text editor; I use emacs.

Assignment 1 Questions

1.1: What do you hope to learn from this course? What skills do you seek to acquire?

1.2: The spec for PSI, the language for which you'll be building an interpreter, is available in Canvas under the Files tab. Skim over it—don't spend more than half an hour. You're not expected to know how to implement all these features yet, but tell me which ones you expect to be most difficult, and why.

Assignment 1 Project

1.3: Download `hueplot.c` from Canvas (Canvas > Files). This is a working C program, and it uses a lot of features you haven't been taught, so you're not expected to understand everything. Spend about half an hour reading the code; understand what you can, but don't worry too much about it—the purpose here is initial exposure. We'll go over in detail in the third office hour.

Compile it:

```
$ gcc -o hueplot hueplot.c
```

And then run it with various arguments:

```
$ ./hueplot power 3
...
$ ./hueplot tripole
...
```

The program will create an image in `out.ppm`, which you should examine. Then tell me:

What does `mystery` do? Try running it for different values, starting small, to get a sense of what it's doing. Don't try values in excess of 1000; it will take too long.

1.4: Write a program that does something—anything. Compile it, run it, and tell me how you verified that it works.